

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №2 (Яндекс.Контест)

Выполнил:

Студент группы Р3210

Давидюк Антон

Задача №1 Е. Коровы в стойла

Разбор: left хранит минимальное расстояние между стойлами, right максимально возможное (это разница между стойлом последним и первым), best_res запоминает наибольшее найденное расстояние. В алгоритме используется бинарный поиск, на каждом шаге вычисляем среднее значение mid. Функция проверяет, можно ли разместить k стойл так, чтобы расстояние между двумя было не меньше mid. Placed считает кол-во уже размещенных стойл, last_pos позиция последнего размещенного стойла. После проходимся по массиву stoylos, если стойло находится на расстоянии $\geq$ mid от последнего размещенного, то размещаем новое и ставим last_pos. Если разместили k или более стойл, значит записываем mid в best_red и пробуем идти дальше (left = mid+1) или уменьшаем расстояние, поиск продолжается пока left <= right.

Алгоритмическая сложность:

1. По времени: выполняется бинарный поиск по диапазону от 0 до $\max(\text{stoylos}) - \min(\text{stoylos})$, количество итераций $O(\log M)$, M это максимальное расстояние, внутри каждой итерации выполняется проход по стойлам за $O(n)$. Сложность $O(n \cdot \log M)$
2. По памяти: вектор stoylos хранит n вводных значений, он занимает $O(n)$ памяти. Переменные занимают константное кол-во памяти. Сложность по памяти $O(n)$

Код:

```
#include <algorithm>
#include <iostream>
#include <vector>

int MaxDist(int k, const std::vector<int>& stoylos) {
    int left = 0;
    int right = stoylos.back() - stoylos.front();
    int best_res = 0;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        int placed = 1;
        int last_pos = stoylos[0];
        bool can_place = false;

        for (size_t i = 1; i < stoylos.size(); ++i) {
            if (stoylos[i] - last_pos >= mid) {
                placed++;
                last_pos = stoylos[i];
                if (placed >= k) {
                    can_place = true;
                    break;
                }
            }
        }

        if (can_place) {
            best_res = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return best_res;
}

int main() {
```

```

int n, k;
if (std::cin >> n >> k) {
    std::vector<int> stoylos(n);
    for (int i = 0; i < n; ++i) {
        std::cin >> stoylos[i];
    }
    std::cout << MaxDist(k, stoylos) << std::endl;
}
return 0;
}

```

Задача №2 F. Число

Разбор: в parts хранятся все части числа, line нужна для чтения строк из потока. Алг читает строки и сохраняет в вектор, после част сортируются с помощью компаратора, он сравнивает строки pfirst и psec, если pfirst + psec больше чем psec + pfirst, то pfirst идет перед psec. Так мы получаем гарантию, что все строки отсортированы в порядке при котором мы получаем максимальное число.

Алгоритмическая сложность:

1. По времени: чтение строк из потока занимает $O(n * L)$, n это кол-во строк, L средняя длина. Сортировка вектора из N элементов с компаратором занимает $O(n * \log(n) * L)$. Сложность по времени $O(n * \log(n) * L)$.
2. По памяти: вектор хранит n строк длиной $O(n * L)$. Line и временные строки занимаю $O(L)$. Сложность по памяти $O(n * L)$

Код:

```

#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

bool comparat(const std::string& pfirst, const std::string& psec) {
    return pfirst + psec > psec + pfirst;
}

int main() {
    std::vector<std::string> parts;
    std::string line;

    while (std::cin >> line) {
        parts.push_back(line);
    }
    std::sort(parts.begin(), parts.end(), comparat);
    for (const auto& part : parts) {
        std::cout << part;
    }
    std::cout << std::endl;
    return 0;
}

```

Задача №3 G. Кошмар в замке

Разбор: в векторе `weight` содержится 26 весов для букв. Вектор `fr` содержит частоту встречаемости каждой буквы в строке. В алгоритме буквы разделяются на группы: `mnogos` (встречающиеся 2 и более раз) и `odins` (буквы, встречающиеся 1 раз). Группы сортируются по убыванию весов компаратором `check`. Финальная строка `res` записывается в 3 этапа: 1 для каждой буквы из `mnogos` размещаем по одному экземпляру в начало `l` и конец `r` строки. 2 оставшиеся буквы из `mnogos` размещаются в центральной части строки. 3: буквы из `odins` размещаются в свободные места в центре. Такая расстановка максимизирует итоговый вес строки.

Алгоритмическая сложность:

1. По времени: подсчет частоты букв занимает $O(n)$, n — длина строки. Сортировка занимает $O(1)$, заполнение финальной строки требует одного прохода по всем позициям. Сложность по времени $O(n)$.
2. По памяти: векторы `weight`, `fr`, `mnogos`, `odins` фиксированного размера, это $O(1)$. Финальная строка занимает $O(n)$ памяти. Сложность по памяти $O(n)$.

Код:

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::string s;
    std::cin >> s;

    std::vector<long long> weight(26);
    for (int i = 0; i < 26; ++i) {
        std::cin >> weight[i];
    }

    int n = static_cast<int>(s.size());
    std::vector<int> fr(26, 0);
    for (char c : s) {
        fr[c - 'a']++;
    }

    std::vector<std::pair<long long, int>> odins;
    std::vector<std::pair<long long, int>> mnogos;

    for (int i = 0; i < 26; ++i) {
        if (fr[i] > 0) {
            if (fr[i] >= 2) {
                mnogos.push_back({weight[i], i});
            } else {
                odins.push_back({weight[i], i});
            }
        }
    }

    auto check = [](const std::pair<long long, int>& a, const std::pair<long long, int>& b) {
        return a.first > b.first;
    };
};
```

```

std::sort(mnogos.begin(), mnogos.end(), check);
std::sort(odins.begin(), odins.end(), check);

std::string res(n, ' ');
int l = 0;
int r = n - 1;

for (size_t i = 0; i < mnogos.size(); ++i) {
    int indx = mnogos[i].second;
    char c = static_cast<char>('a' + indx);

    res[l] = c;
    res[r] = c;
    l++;
    r--;
}

for (size_t i = 0; i < mnogos.size(); ++i) {
    int indx = mnogos[i].second;
    int count = fr[indx] - 2;
    char c = static_cast<char>('a' + indx);

    for (int j = 0; j < count; ++j) {
        res[l] = c;
        l++;
    }
}

for (size_t i = 0; i < odins.size(); ++i) {
    int indx = odins[i].second;
    char c = static_cast<char>('a' + indx);
    res[l] = c;
    l++;
}

std::cout << res << std::endl;
return 0;
}

```

Задача №4 Н. Магазин

Разбор: алгоритм сортирует вектор `price` по убыванию с помощью `greater<int>()`, самые дорогие товары оказываются в начале, `total` накапливает финальную сумму к оплате, `indx` отслеживает порядок товара в отсортированном списке. Цикл проходится по всем товарам, если `indx % k != 0`, то цена добавляется в `total` иначе товар пропускается.

Алгоритмическая сложность:

1. По времени: чтение n товаров из потока занимает $O(N)$. Сортировка и вектора из n элементов занимает $O(n \cdot \log n)$, проход по отсортированному вектору занимает $O(n)$. Сложность по времени $O(n \cdot \log n)$
2. По памяти: вектор `price` хранит n цен и занимает $O(n)$. Переменные для логики занимают $O(1)$. Сложность по памяти $O(n)$

Код:

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

long long calculate(vector<int>& price, int k) {
    sort(price.begin(), price.end(), greater<int>());

    long long total = 0;
    int indx = 1;
    for (int pri : price) {
        if (indx++ % k)
            total += pri;
    }
    return total;
}

int main() {

    int n, k;
    cin >> n >> k;

    vector<int> price(n);
    for (int i = 0; i < n; i++) {
        cin >> price[i];
    }

    long long res = calculate(price, k);
    cout << res << endl;

    return 0;
}
```